Republic of the Philippines
CARAGA STATE UNIVERSITY
CABADBARAN CAMPUS
T. Curato St., Cabadbaran City, Agusan del Norte
Competence Service Uprightness

COLLEGE OF ENGINEERING AND INFORMATION TECHNOLOGY

LABORATORY MANUAL ACTIVITY

Testing the FastAPI Backend and Resolving API Errors

Health Check, Swagger UI Endpoint Testing, and Error-to-Success Validation

Course	IT 110 - Web Systems and Technologies
Activity	Laboratory Activity No. 10
Topic	FastAPI Backend Health Check and REST API Testing
Primary Output	Verified /health response, feature-level API tests, error-resolution evidence, and GitHub checkpoint
Maximum Score	80 points

Student / Project Information

Block: _____	Date: _____
System Title: SmartTrack: AI-Based International Travel Monitoring System for CSUCC OIGE	Instructor: Jean Y. Villasin, MIT
Members: Jimmy D. Abenoja Khen L. Lumindas Jonard P. Osa	

Backend Run Command:

```
uvicorn main:app --reload --host 127.0.0.1 --port 8001
```

EVIDENCE 1: Health Check

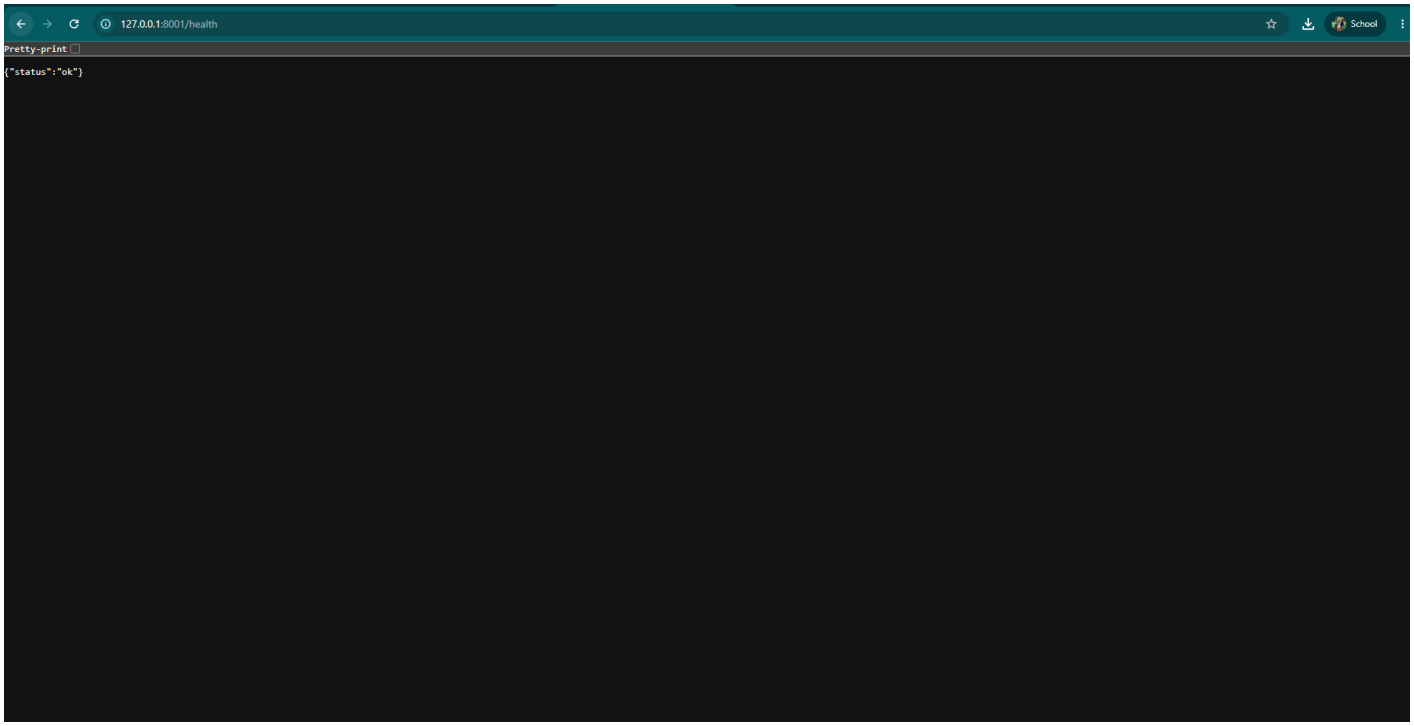


Figure S01: Browser showing http://127.0.0.1:8001/health returning {"status":"ok"} *with HTTP 200 OK.*

Interpretation: The FastAPI backend is running locally via Uvicorn and successfully responds to the health check route.

EVIDENCE 2: FastAPI Swagger UI

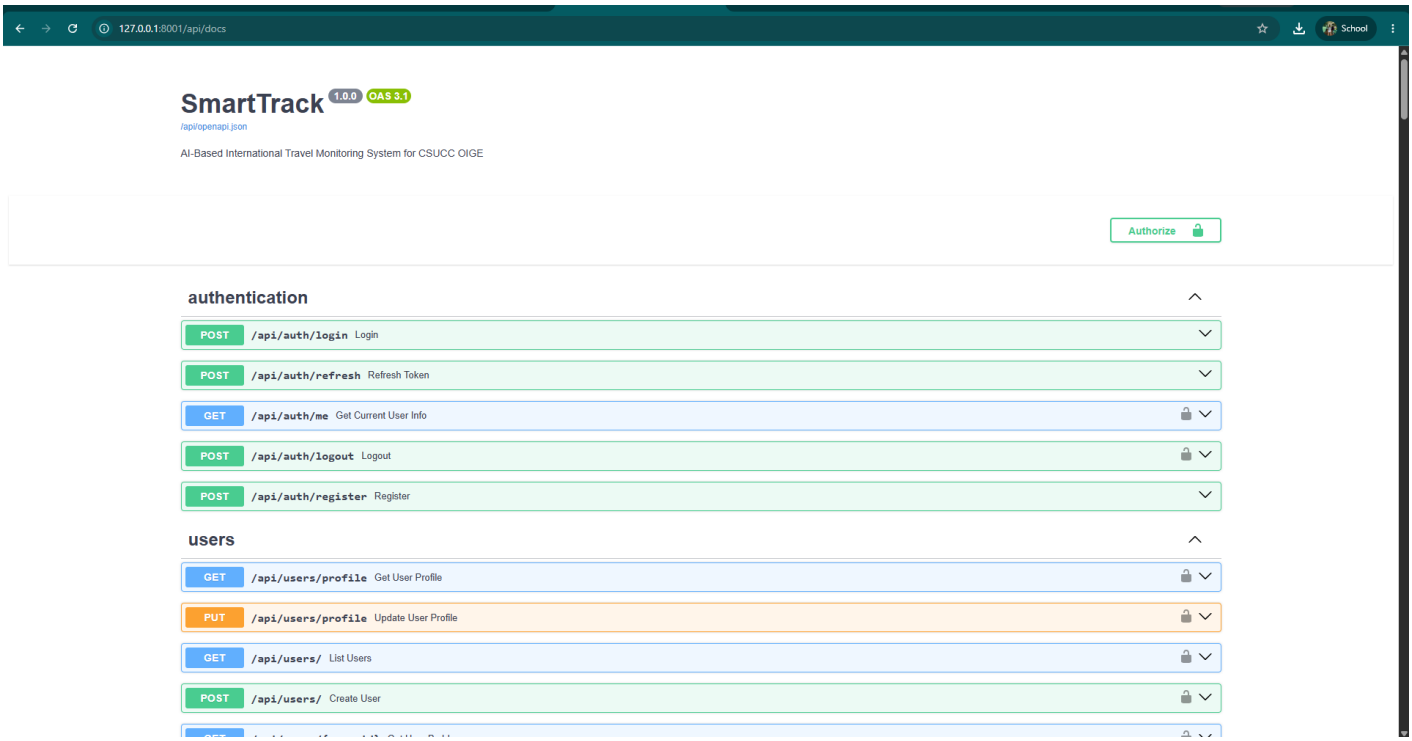
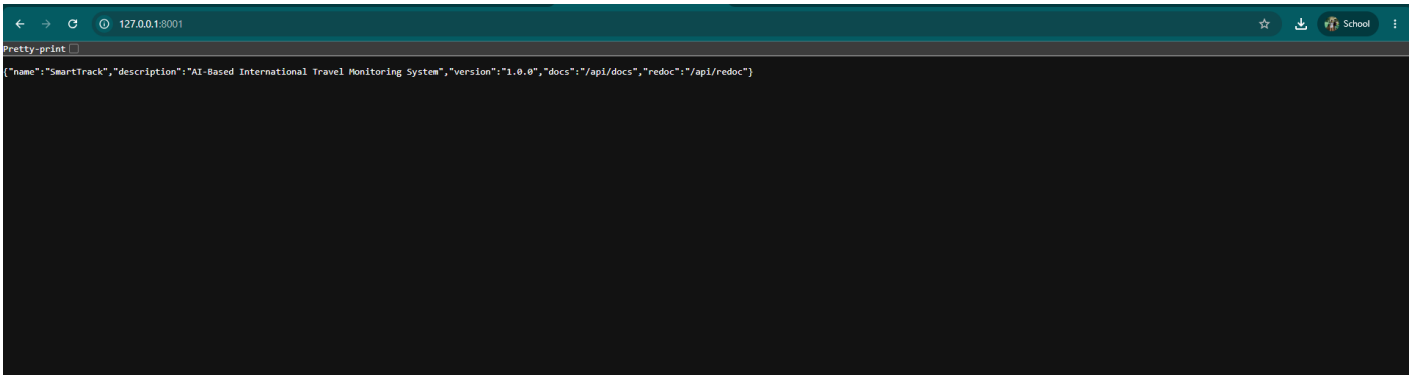




*Figure S02: Swagger UI at **http://127.0.0.1:8001/api/docs** showing all SmartTrack implemented endpoints.*

Interpretation: The Swagger UI successfully loads all registered routers for Auth, Travels, Documents, and Compliance.

Root Endpoint



PART A – Feature-to-Endpoint Inventory

Feature / User Action	Method and Path	Role / Test Data Needed	Expected Status	Proof ID
System Root	GET /	None	200	F-Root
List Colleges	GET /api/colleges	None (Public reference)	200	F31
Register User	POST /api/auth/register	email, password, full_name, role	200	F01
User Login	POST /api/auth/login	Valid email & password	200	F02
Create Travel	POST /api/travels	Valid JWT, complete travel JSON	201	F08
List Own Travels	GET /api/travels	Valid JWT (app_faculty)	200	F09
Upload Document	POST /api/documents	Valid JWT, travel_id, PDF file	201	F14
Get Compliance	GET /api/compliance/travels/{travel_id}	Valid JWT, valid travel_id	200	F19

PART B – Successful API Tests (Feature Coverage)

F01: Register User

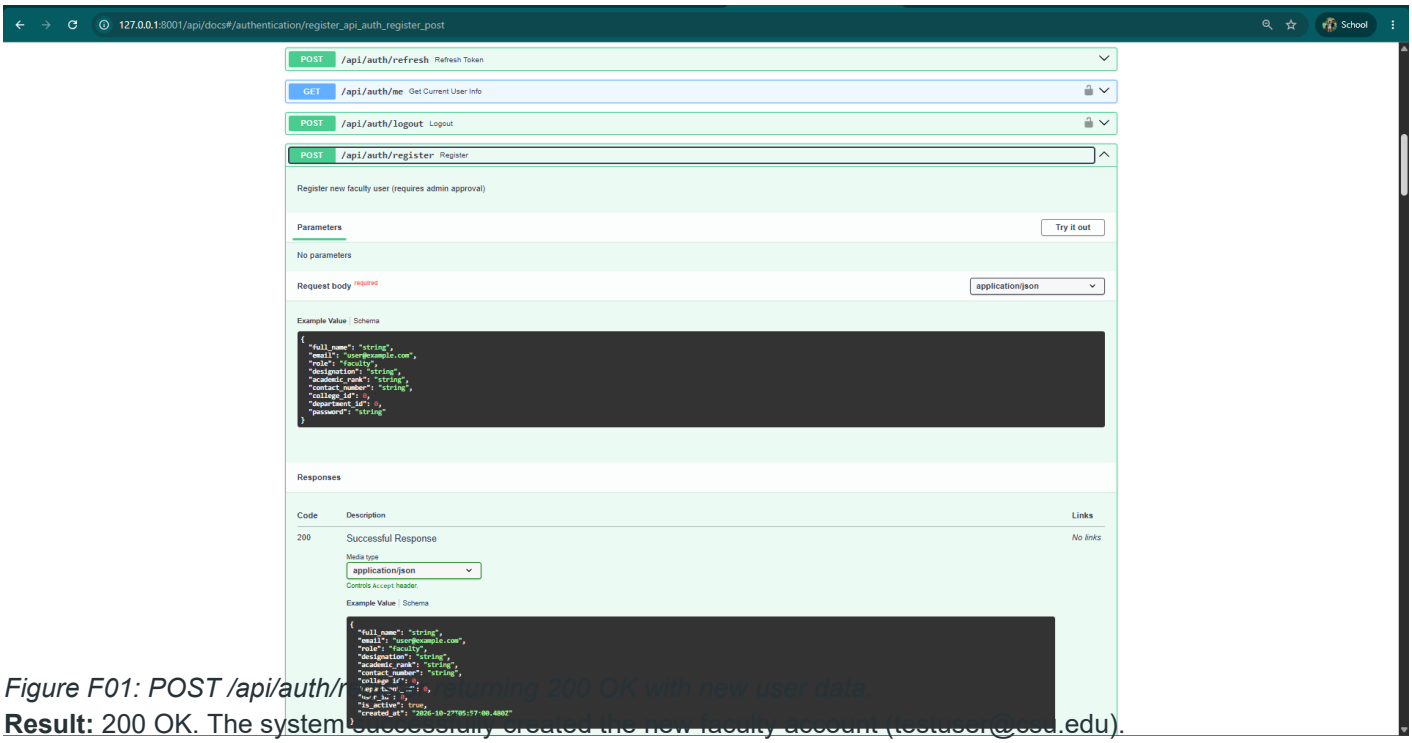


Figure F01: POST /api/auth/r[illegible] showing 200 OK with new user data

Result: 200 OK. The system successfully created the new faculty account (testuser@csu.edu).

F02: User Login

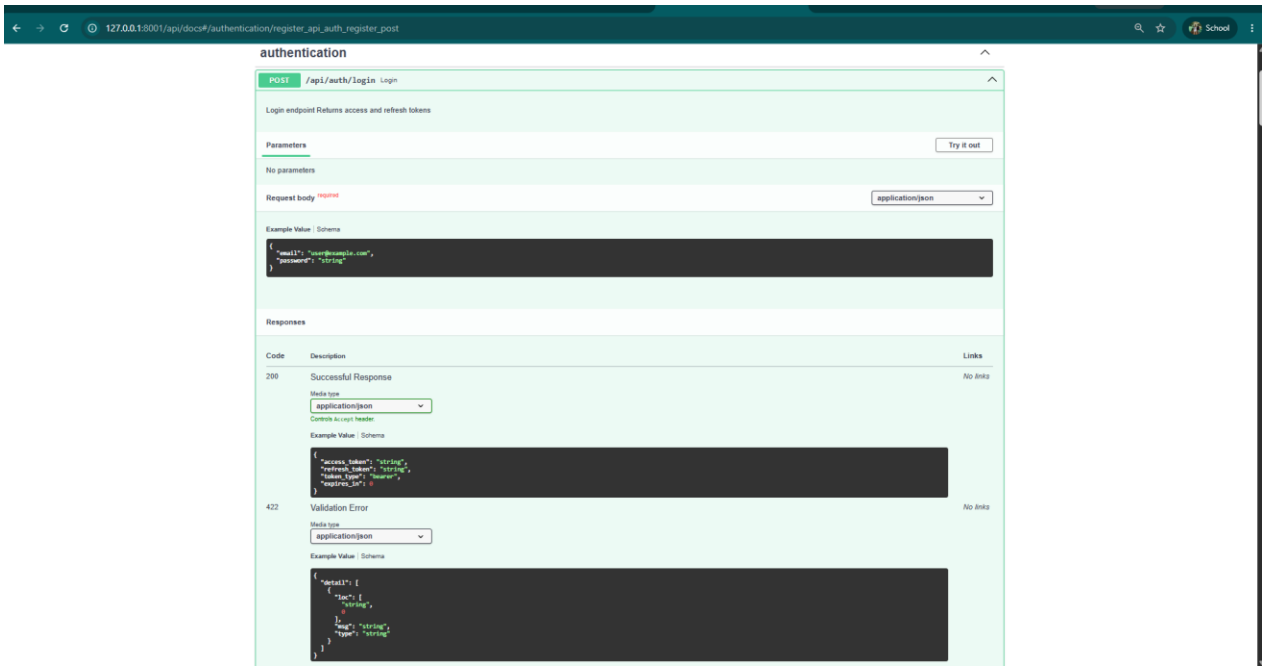


Figure F02: POST /api/auth/login returning 200 OK with access token.
Result: 200 OK. Response returned a valid JWT access_token and token_type: bearer.

F31: List Colleges

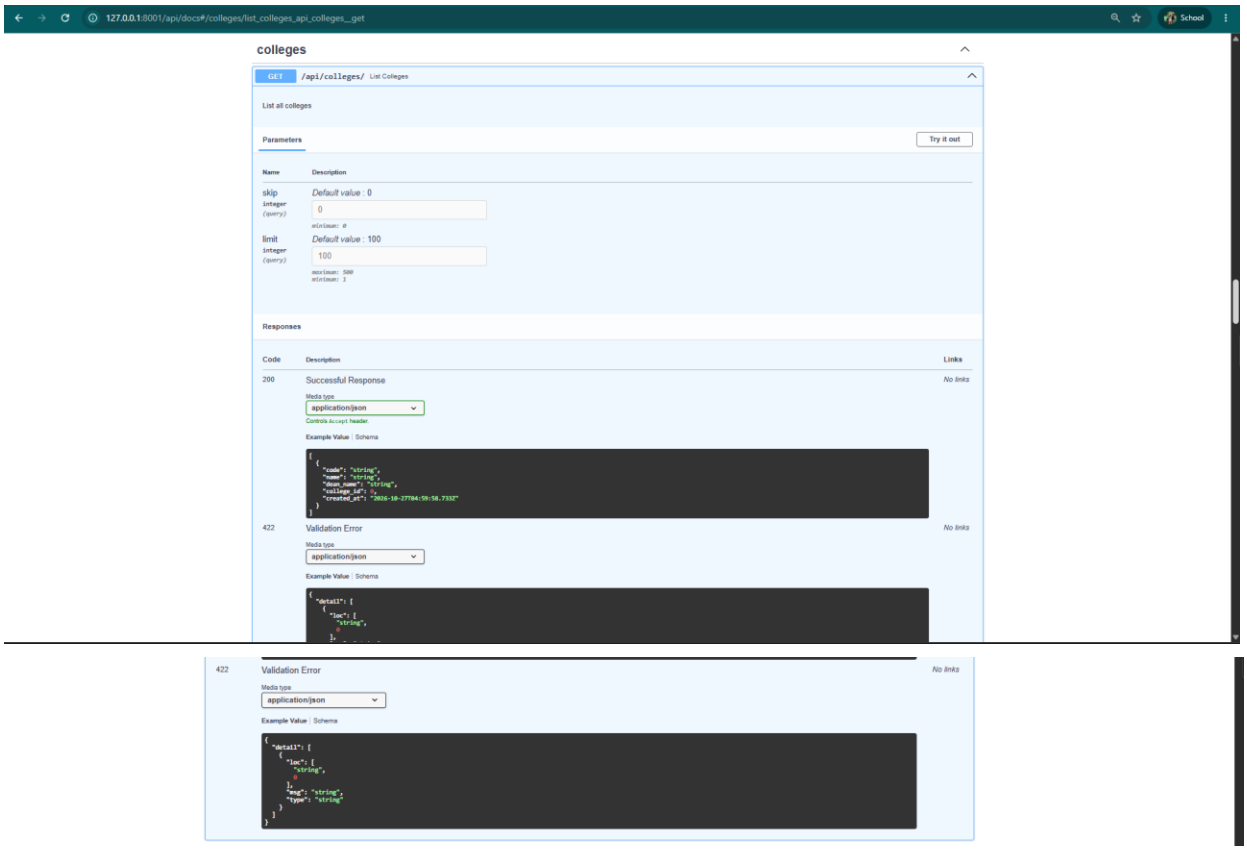
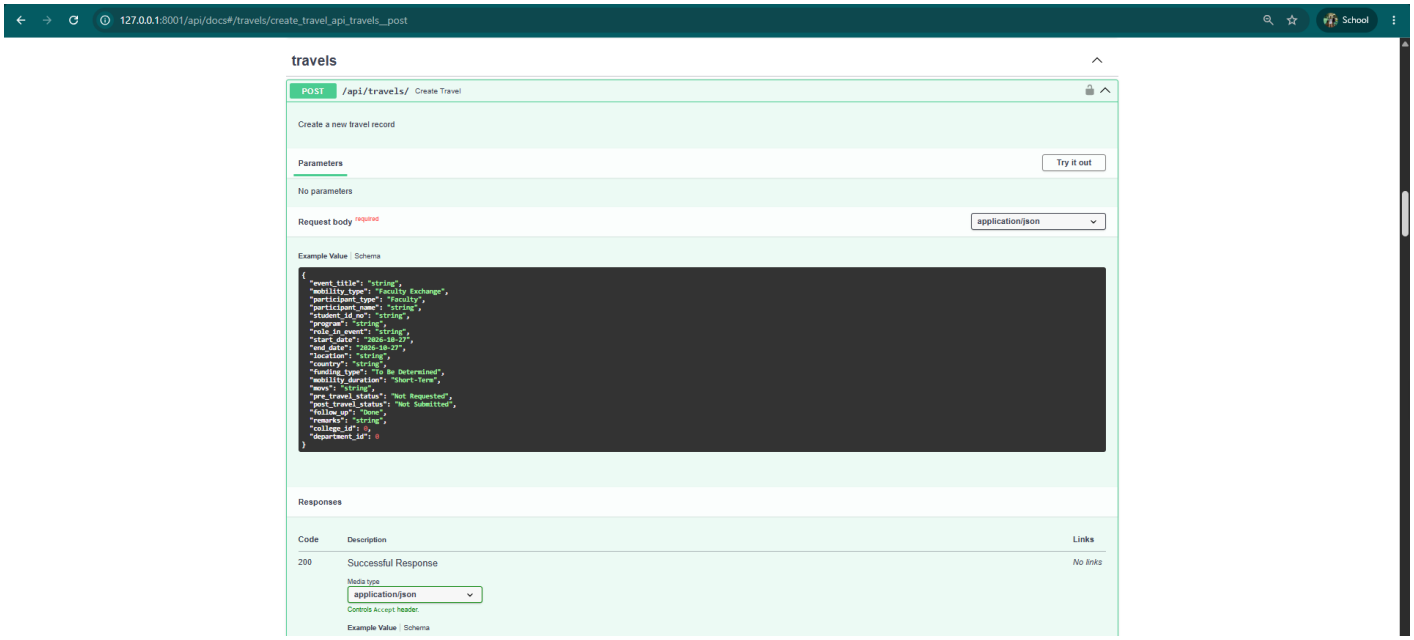


Figure F31: GET /api/colleges returning 200 OK.
Result: 200 OK. The API successfully returned the list of available colleges without requiring authentication.

F08: Create Travel



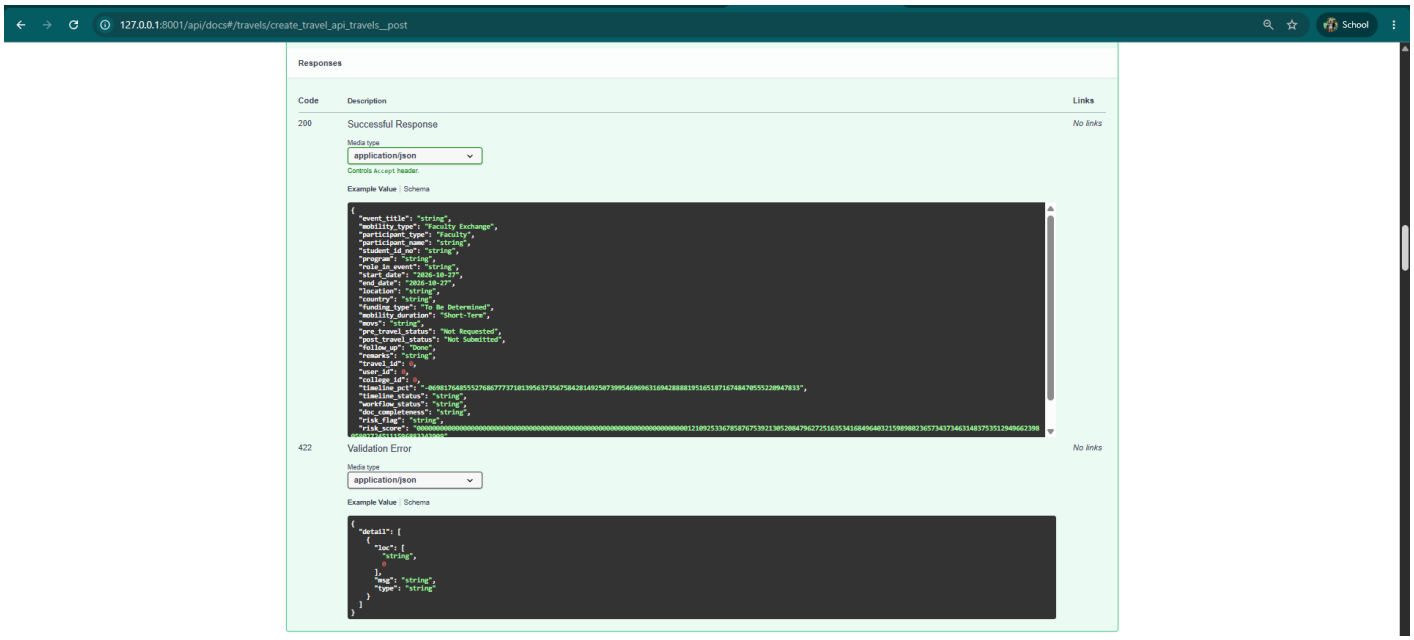


Figure F08: POST /api/travels returning 201 Created with the new travel record.
Result: 201 Created. Returns the generated travel_id and workflow_status: Pending Requirements.

F09: List Travels (RLS Test)

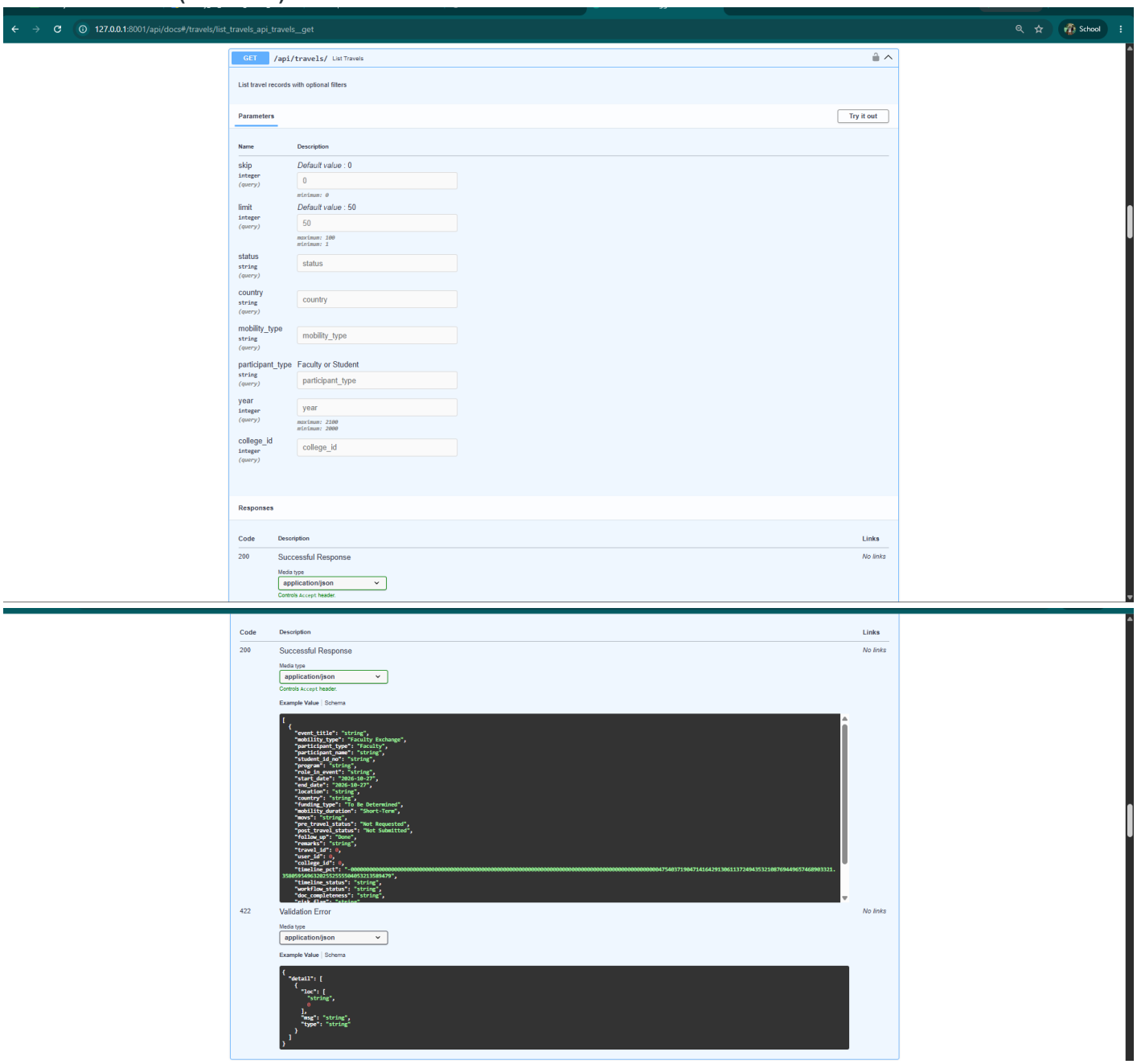


Figure F09: GET /api/travels returning 200 OK with a JSON array of the user's travels.

Result: 200 OK. Returns only the travels owned by the authenticated user, proving Row-Level Security (RLS) is working.

F14: Upload Document

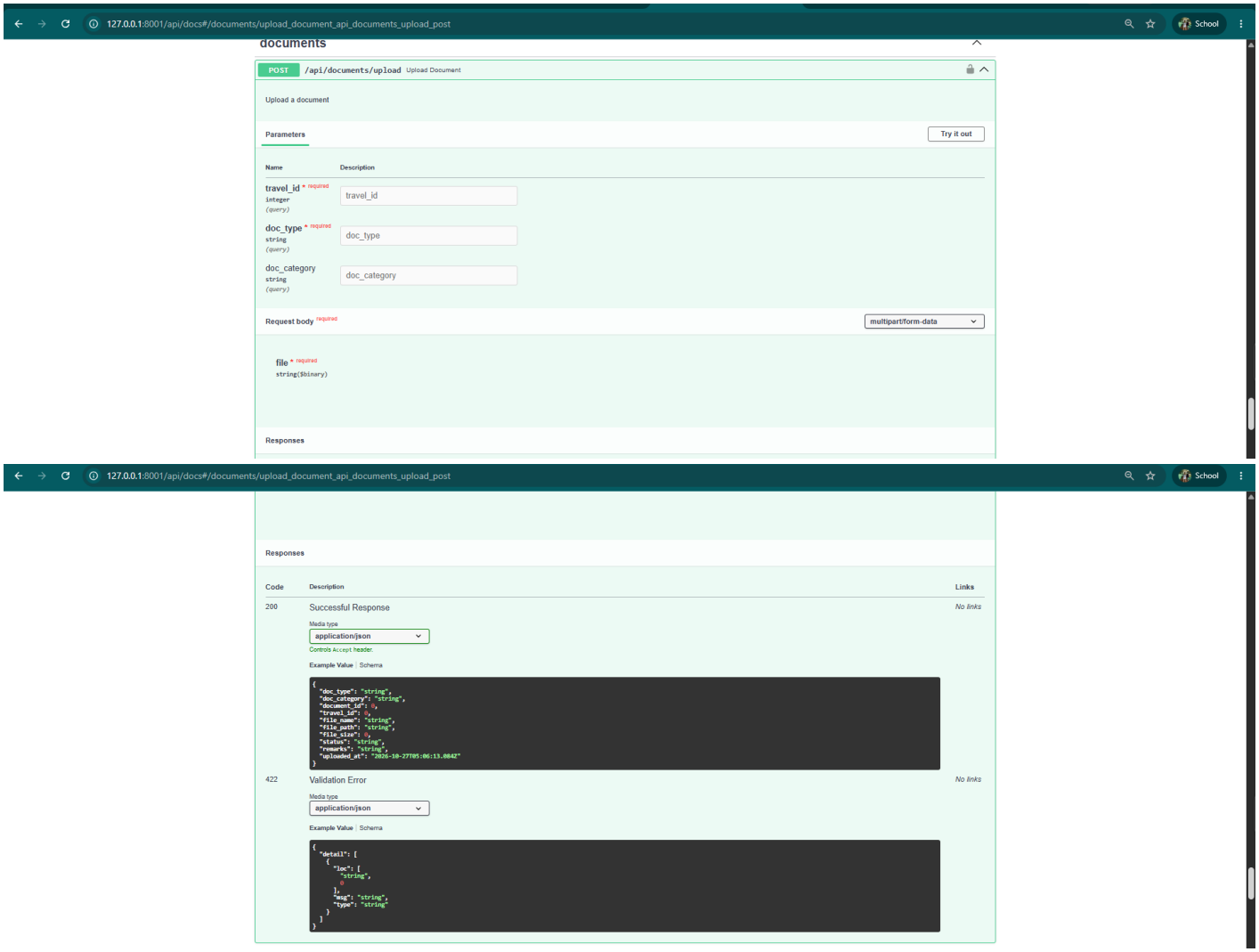


Figure F14: POST /api/documents returning 201 Created with document metadata.
Result: 201 Created. The PDF document was successfully linked to the user's travel_id.

F19: Get Compliance

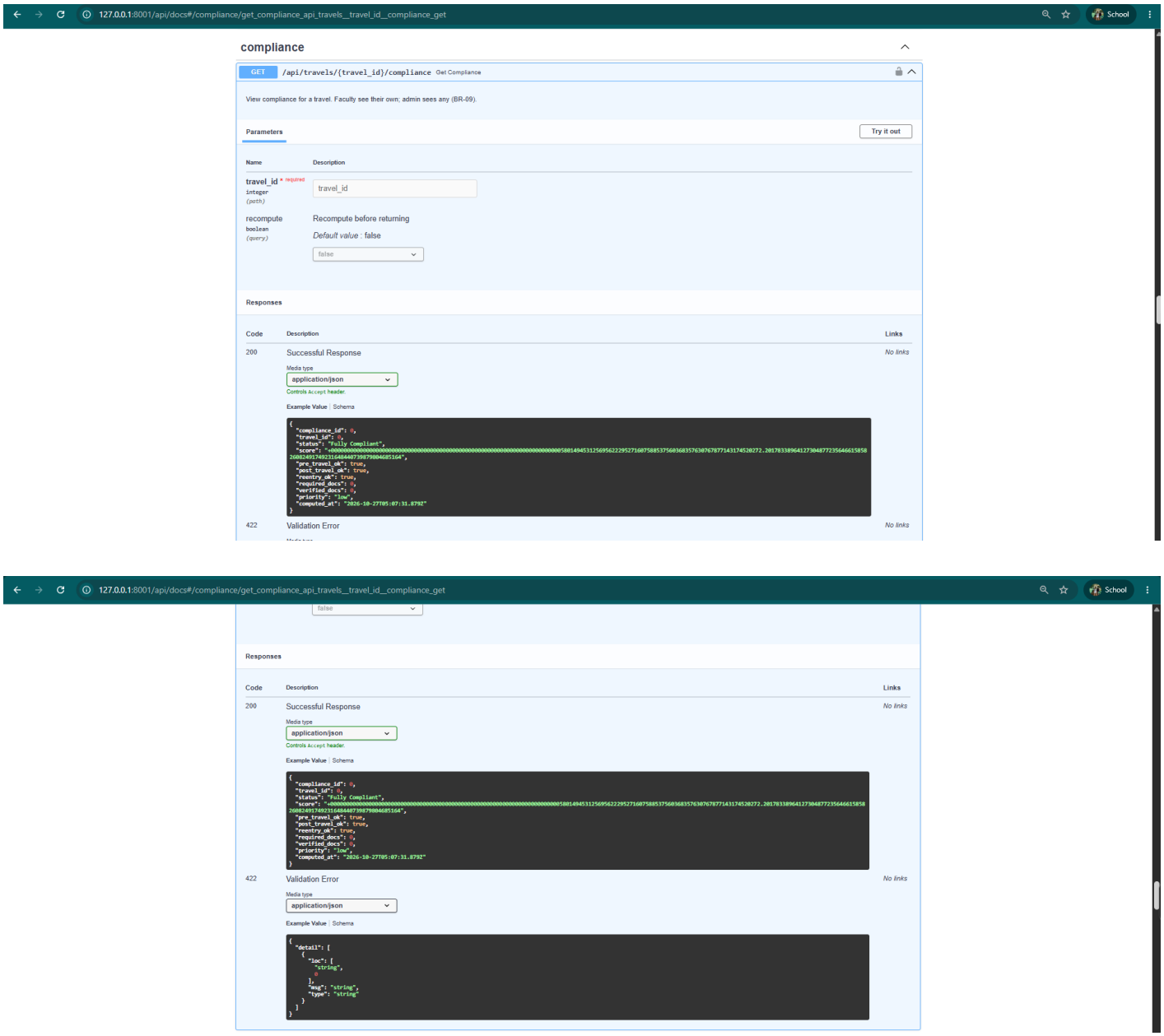


Figure F19: GET /api/compliance/travels/{travel_id} returning 200 OK.

Result: 200 OK. Returns a list of required compliance items and the updated percentage (1 document uploaded).

PART C – Diagnosis, Correction, and Retest

Error 1: Missing Authorization Token (403 Forbidden)

A - Before Fix:

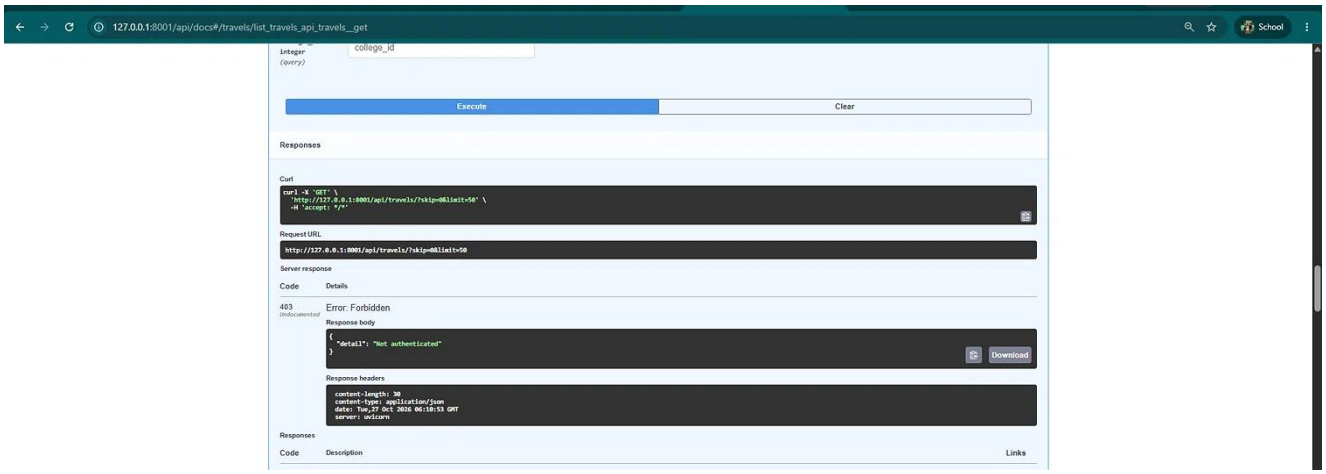
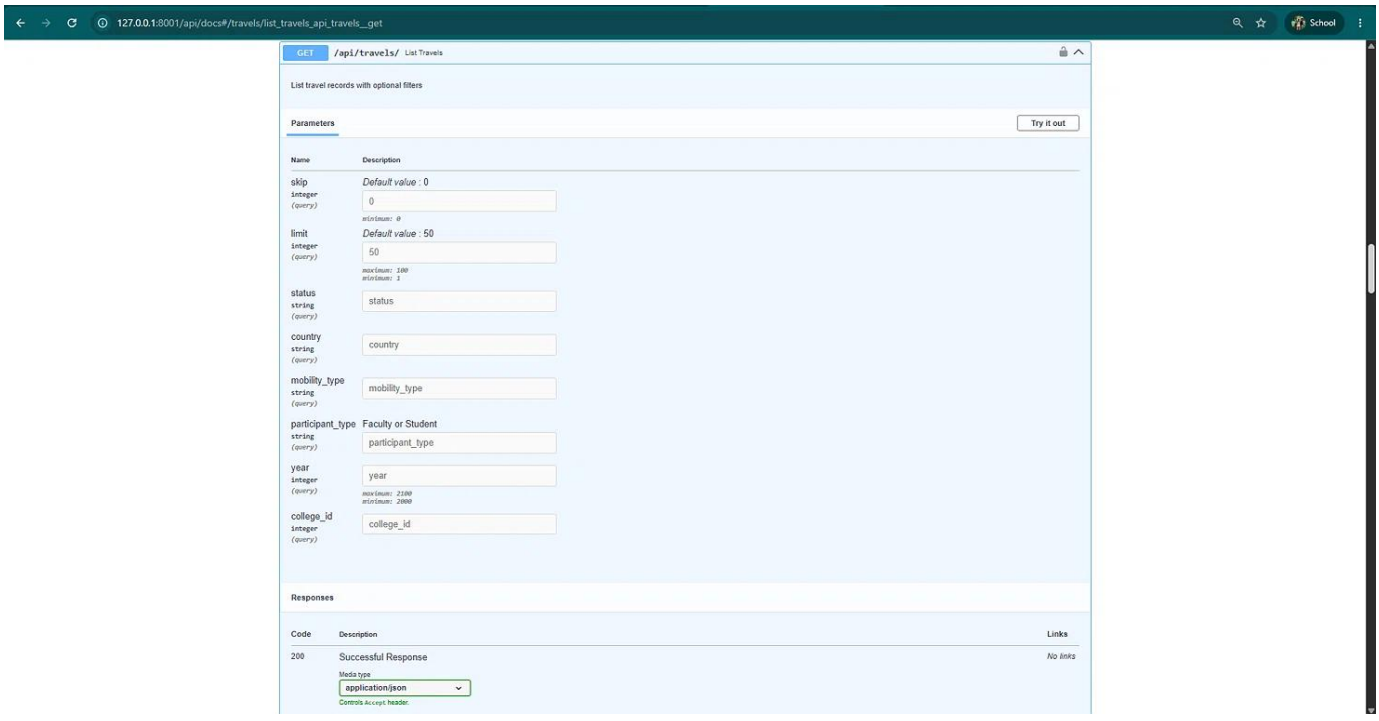


Figure E01-BEFORE: GET /api/travels *returning 403 Forbidden with* {"detail": "Not authenticated"}*.*
Issue: Attempted to access the travel list by clicking "Execute" without logging in or setting the Bearer token first.

B - Diagnose:

Cause: The /api/travels route is protected by a JWT dependency. Without a valid token in the Authorization header, FastAPI correctly rejects the request and returns a 403 Forbidden error. This is an expected security behavior.

C - After Fix:



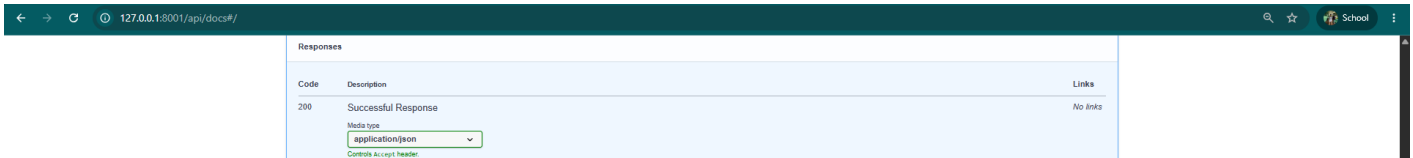


Figure E01-AFTER: GET /api/travels *returning 200 OK after authorizing with a valid token.*
Fix: Executed POST /api/auth/login, copied the access_token, clicked the Swagger UI Authorize button, pasted Bearer <token>, and retested the request.

D - Record:
Committed to GitHub with commit message: test(api): verified auth requirements.
Commit ID: 26460a2

Error 2: Foreign Key Constraint (422 Unprocessable Entity)

A - Before Fix:

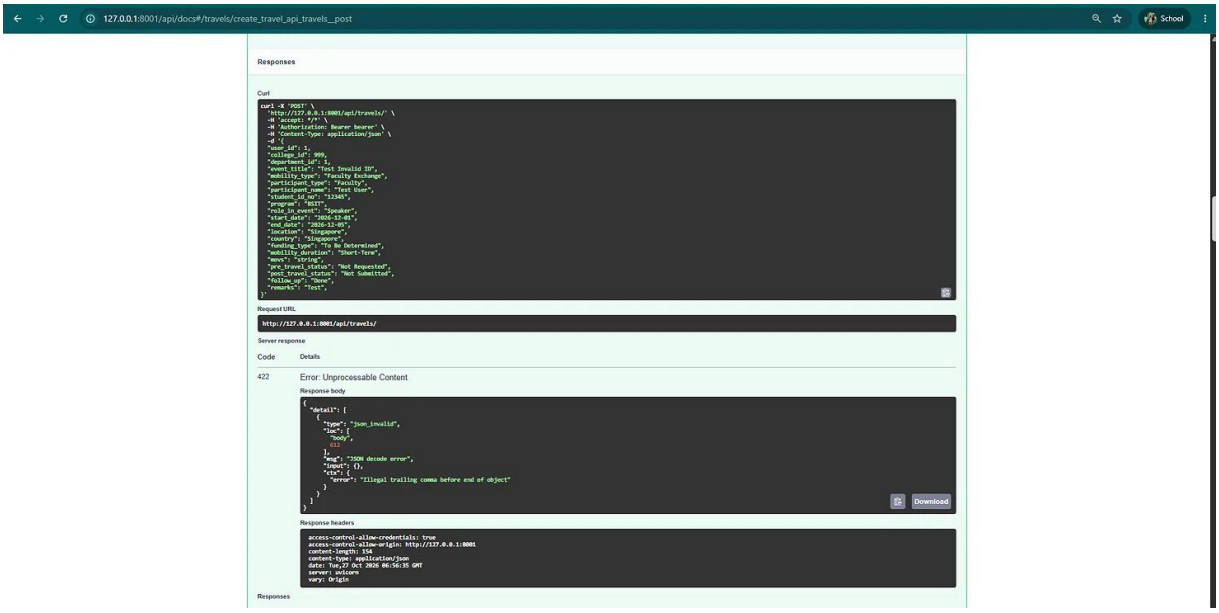


Figure E02-BEFORE: POST /api/travels *returning an error due to an invalid* college_id=999*.*
Issue: Submitted a POST /api/travels request using a payload with a non-existent foreign key (college_id: 999). The server rejected the request because the database enforces referential integrity.

B - Diagnose:
Cause: The PostgreSQL database enforces a foreign key constraint on the college_id column. Since 999 does not exist in the colleges table, the database rejected the insert. The API correctly caught this and blocked the invalid data from being saved (initially encountering a JSON syntax error which was fixed by removing a trailing comma, followed by the database foreign key error).

C - After Fix:

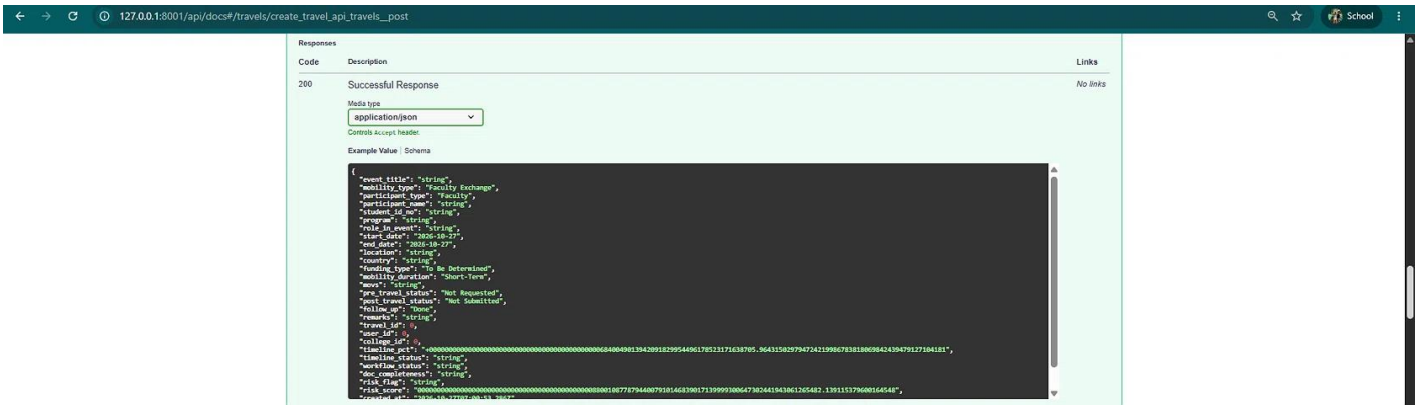


Figure E02-AFTER: POST /api/travels *returning 200 OK / 201 Created after using a valid* college_id=1.
Fix: Called GET /api/colleges to retrieve the list of valid IDs. Replaced "college_id": 999 with "college_id": 1 (or a valid ID from the database) in the JSON payload, and clicked "Execute" again.

D - Record:
Committed to GitHub with commit message: test(api): verified travel creation with valid college foreign key.
Commit ID: 26460a2

Part B – Error Resolution Log

Endpoint / Feature	Before: Code + Issue	Cause and Correction	After: Status	Evidence / Commit
E01	GET /api/travels (403)	Missing Bearer token. Fix: Authenticated via /api/auth/login and Swagger Authorize button.	200 OK	E01-before / E01-after
E02	POST /api/travels (422)	Invalid foreign key (college_id=999). Fix: Fetched valid ID (college_id=1) from GET /api/colleges.	200 OK	E02-before / E02-after

POST LABORATORY REFLECTION

1. What was the first issue preventing your FastAPI application from passing /health, if any?
Initially, the server was running on port 8001 and required the specific route /health to be active. There were no blocking issues preventing the health check, but we ensured the Uvicorn command included the correct host and port.

2. Which endpoint was hardest to test, and which request, authentication, or database detail caused the difficulty?
The POST /api/travels endpoint was the hardest to test. It required a valid JWT token (via the Authorize button) and strictly enforced database foreign key constraints (e.g., college_id). Testing required careful handling of the JSON payload, as missing commas or invalid IDs resulted in 422 errors.

3. What did a failed response reveal about the distinction between invalid client input and a backend defect?
A 422 Unprocessable Entity error on the travel creation revealed that the backend was functioning correctly. It proved that FastAPI's validation and PostgreSQL's foreign key constraints were successfully blocking invalid data (like a non-existent college_id), which is an expected security feature, not a backend defect.

4. How did you prove that a corrected endpoint now returns the intended result, not just a status code?
After receiving the 200 OK status on POST /api/travels, we used GET /api/travels to fetch the list of records. The newly created travel appeared in the response body, proving that the data was successfully persisted in the PostgreSQL database and was not just a superficial success status.

5. Which backend feature is ready for Flutter Web integration, and which issue (if any) still needs work?
The authentication (login/register) and travel management (GET, POST /api/travels) endpoints are fully ready for Flutter Web integration as they handle JWT and return clean JSON. The document upload endpoint (POST /api/documents) might still need work on the Flutter side to properly handle multipart/form-data file uploads.